

Unidad 4

Calidad del producto

La calidad del producto es el grado en el que el software cumple con los requerimientos funcionales y no funcionales y las expectativas del cliente o usuario final.

Un producto de calidad no solo funciona sino que funciona bien (rendimiento, interface, etc)

Técnicas y herramientas para asegurar la calidad del producto

En el proyecto:

- Revisiones técnicas
- Auditorías

En el producto

- Aseguramiento de calidad (QA → Asegura que se cumplan los estándares de calidad definidos)
 - Usa revisiones técnicas
 - Auditorías de configuración
- Control de calidad (QC → Se enfoca en detectar defectos en el producto)
 - Se hace principalmente con testing

Revisiones técnicas de calidad (QA)

Son el **método principal para validar la calidad** de un proceso o producto.

Un grupo examina el producto o el proceso para **encontrar problemas o incumplimientos**.

Tipos de revisiones:

1. **Inspecciones** → buscan **remover defectos** antes de que lleguen al usuario.
2. **Revisiones de progreso** → revisan si el trabajo avanza según lo planificado.
3. **Revisiones de calidad** → verifican si el producto cumple con los **estándares** definidos.

Puntos claves sobre la calidad del producto

La calidad **se analiza tanto en el proceso como en el producto**.

La **calidad del software es difícil de medir**, porque intervienen muchos factores (usabilidad, rendimiento, seguridad, etc.).

El proceso de desarrollo (cómo se trabaja) es la base para mejorar la calidad del producto.

Trabajar con calidad es más barato que hacerlo sin calidad, porque evita retrabajos, errores y pérdida de tiempo.

Mejorar los procesos de calidad **genera grandes beneficios económicos** a largo plazo (retorno de inversión alto).

Testing en ambiente ágiles

Es una forma moderna y continua de probar software dentro de los equipos que usan metodologías ágiles.

A diferencia del enfoque tradicional no se prueba al final sino que durante todo el desarrollo se hace. Se busca asegurar la calidad desde el inicio.

Manifiesto del testing en Agil

Se basa en 5 principios fundamentales que cambian totalmente la mentalidad del tester

- Pruebas durante sobre pruebas al final
- Prevenir errores sobre encontrar errores
- Entender lo que se prueba sobre solo entender las funcionalidad
- Construir un mejor sistema sobre romper el sistema
- Responsabilidad de todo el equipo por la calidad (No es tarea de una sola persona sino de todo el equipo).

Enfoque tradicional VS enfoque agil

Enfoque Tradicional	Enfoque Ágil
Secuencial (Cascada) – las fases van una después de otra.	Iterativo e incremental – se repite en ciclos cortos (sprints).
El testing es una fase separada al final.	El testing es una actividad integrada en todo el desarrollo.
Feedback lento (meses o años).	Feedback rápido y constante (días o semanas).
El tester prueba luego de terminar el código .	El tester participa desde el inicio con el equipo.

Principios y prácticas del testing ágil (9 principios claves)

- ✓ El testing **se adelanta en el proyecto** (desde los primeros días).
- ✗ **No es una fase**, sino una **actividad continua**.
- 👥 **Todos hacen testing**, no solo el área QA.
- ⚡ Se busca **reducir el tiempo de feedback** (detectar rápido los errores).
- 🎯 Las pruebas se basan en **criterios de aceptación** claros.
- 🔧 Mantener el código limpio y **corregir defectos rápido**.
- 📄 Menos documentación, más colaboración.
- ✓ Las pruebas forman parte del **“Definition of Done”** (terminar no es solo programar, es probar).

 Se pasa de “probar al final” a **desarrollar guiado por pruebas** (*Test Driven Development*).

6 prácticas concretas

Práctica	Qué hace
 TDD (Test Driven Development)	Primero se crean pruebas unitarias, después el código. Se repite el ciclo “Prueba → Código → Refactorizar”.
 ATDD (Acceptance Test Driven Development)	Se crean pruebas de aceptación junto con el PO, antes del desarrollo. Guían el desarrollo.
 Pruebas automatizadas	Unitarias e integración: rápidas, repetibles, aseguran estabilidad.
 Pruebas de regresión automatizadas	Se ejecutan cada vez que se hace un cambio, para comprobar que nada se rompió.
 Pruebas exploratorias	Pruebas manuales creativas sin pasos fijos: buscan descubrir lo inesperado.
 Control de versiones de pruebas	Las pruebas se guardan y versionan junto con el código.

Rol del tester ágil

El tester **no trabaja solo**, sino integrado al equipo.
Colabora con:

- **Product Owner (PO):**
 - Ayuda a definir historias de usuario y criterios de aceptación.
 - Entiende el negocio para probar con sentido.
- **Developers:**
 - Diseña estrategias de prueba.
 - Automatiza pruebas (TDD / ATDD).
 - Da feedback temprano sobre el código.
- **System Team:**
 - Prepara y mantiene el ambiente de pruebas.
 - Valida la *Definition of Done*.
 - Genera métricas de calidad.

Pirámide del testing ágil

Pirámide Tradicional	Pirámide Ágil
Poquitas pruebas unitarias (1–5%).	Muchas pruebas unitarias (80–90%).
Muchas pruebas GUI lentas.	Pocas pruebas GUI, solo flujos críticos.
Foco en encontrar errores.	Foco en prevenir errores.
Testing costoso y lento.	Testing rápido, automatizado y eficiente.

La automatización del testing

La automatización del testing es clave en ágil y proporciona las siguientes ventajas

- Son de ejecución rápida
- Reutiliza scripts
- Abarca más casos de prueba
- Reduce el error humano
- Permite pruebas continuas
- A largo plazo mejora calidad y baja costos

¿Qué es el Testing?

Es una actividad que intenta detectar defectos en el software, los cuales se asumen como presentes siempre.

El objetivo es identificar defectos. Un testing exitoso es aquel que encuentra defectos (cuya presencia se asume en el software).

Es una actividad de control; se realiza una comparación contra algo (requerimientos, especificaciones). Requiere el nivel de detalle más fino posible.

El testing no asegura la calidad, sino que evidencia defectos. Es imposible asegurar la calidad total mediante testing.

Cuando el testing es suficiente?

El testing exhaustivo (probar todas las combinaciones posibles) es una práctica inviable por ello a la hora de hacer pruebas se definen métodos y estrategias para seleccionar la menor cantidad de pruebas que cubran la mayor cantidad de funcionalidades.

Defecto vs Error vs Falla

Error → Se detecta y corrige en la misma etapa donde se introdujo (se detecta en diseño y se corrige en diseño)

Defecto → Se detecta en etapas o fases diferentes (se introduce en implementación y se encuentra en pruebas)

El defecto tiene severidad y priorización

La severidad es la gravedad del defecto

- Bloqueante → Impide continuar con las pruebas.
- Grave → Funcionalidad principal no opera.
- Leve → Funcionalidad secundaria no opera.
- Cosméticas → Error visual, de texto, etc.

La prioridad define el impacto del defecto en la funcionalidad para el negocio

- Urgencia
- Media
- Baja

Falla: Mal funcionamiento del sistema visible por el usuario

Procesos de pruebas

El testing es un proceso definido por lo que está compuesto en etapas detalladas:

1. **Planificación de pruebas:** Se define que se va a probar cómo y con qué recursos, se establecen los objetivos, los criterios de aceptación, el cronograma los ambientes de prueba y las herramientas el artefacto de esta etapa es el plan de pruebas.
2. **Diseño de casos de prueba:** Se crean los casos de pruebas, cada uno tiene entrada procedimiento y resultado, buscan cubrir los requisitos funcionales y no funcionales.
3. **Ejecución de pruebas:** Se ejecutan los casos de prueba
4. **Evaluación y reportes:** Cada defecto encontrado se documenta, se comparan con los resultados esperados, los desarrolladores lo revisan y el tester lo vuelve a re-verificar.
5. **Informe final:** Se resume qué se probó, cuántos defectos hubo, cuántos se corrigieron, qué partes quedaron pendientes y la evaluación final del producto.

Artefactos

Plan de pruebas → Qué, cómo y quién prueba.

Casos de prueba → Instrucciones detalladas de cada prueba.

Reporte de incidentes → Errores hallados.

Informe final → Resultado global del testing.

Niveles de pruebas y entornos

Niveles de pruebas

1. **Pruebas unitarias:** Las hace el desarrollador y están incluidas en el definition of done. Son pruebas realizadas sobre componentes aislados, es decir, de manera independiente al resto, Se hacen teniendo acceso al código fuente y se suelen reparar los errores apenas se encuentran sin registrarlos formalmente.
2. **Pruebas de integración:** Las realiza el testing y tiene como propósito verificar el funcionamiento de dos o más componentes juntos que se relacionen entre ellos.
3. **Pruebas de sistema:** La realiza el testing y se prueba el sistema como un todo, de forma integral.
4. **Pruebas de aceptación de usuario (UAT):** Las realiza el usuario y se llevan a cabo en el despliegue tienen como finalidad verificar que se cumple con lo que el usuario solicitó.

Ambientes y entornos

1. **Desarrollo:** Es un entorno en donde los desarrolladores realizan pruebas unitarias
2. **Prueba:** Es un entorno en donde se realizan las pruebas de integración y de sistema
3. **Pre-Producción:** Es un entorno en donde se realizan las pruebas de aceptación de usuario (UAT).
4. **Producción:** El ambiente real del cliente

Tipos de pruebas (funcional y no funcional)

Testing funcional

Comprueba que el software haga lo que debe hacer según los requisitos funcionales, se basa en la caja negra (solo importa el resultado no el código), busca responder ¿Hace lo que el cliente pidió?

Testing No funcional

Evalúa cómo funciona el sistema más allá de lo que hace:

Tipos de testing no funcionales:

- Prueba de estrés: Sobrepasar la capacidad máxima del sistema para ver cómo responde el software y hardware
- Prueba de carga: Se lleva a la capacidad máxima pero sin exigir de más para ver cómo responder el software y hardware
- Prueba de rendimiento: Rapidez. tiempos de respuesta
- Prueba de seguridad: Protección contra ataques o accesos indebidos.
- Prueba de usabilidad: Facilidad de uso para el usuario final
- Prueba de compatibilidad: Usabilidad en diferentes sistemas operativos/navegadores etc
- Pruebas de instalación: si el sistema se instala o actualiza correctamente

Otro tipo de prueba:

Smoke test: Es una prueba rápida y superficial que se hace apenas se arma una nueva versión del sistema. Verifica que la básico funcione, no busca detectar errores específicos sino detectar fallas graves.

